Keyword Extraction Project Report

Keyword Extraction from IMDB Reviews

1. Project Overview

The project implements an end-to-end keyword extraction system for IMDB movie reviews. Keyword extraction is the process of identifying the most important words or phrases that summarize the main topic of a document. In this project, each movie review is treated as one document, and the system extracts the most representative keywords for that review.

The project includes text preprocessing, TF-IDF keyword extraction, Word2Vec semantic representation, cosine similarity re-ranking, KMeans clustering, TextRank as a bonus model, evaluation metrics, visualization outputs and a Tkinter graphical user interface.

2. Project Objectives

- Clean and prepare raw movie review text for NLP processing.
- Extract meaningful keywords from each review using TF-IDF.
- Improve keyword extraction using semantic similarity with Word2Vec.
- Use KMeans clustering to produce more diverse keyword sets.
- Compare different keyword extraction methods using precision, recall, and F1-score.
- Provide charts and a GUI to make the system easier to test and demonstrate.

3. Dataset

The project uses the IMDB movie reviews dataset. The full dataset contains movie review texts with sentiment labels. In the keyword extraction task, each review is processed independently so the model can identify the important terms that describe that review.

Item	Description
Dataset	IMDB Movie Reviews Dataset
Number of reviews	50,000 reviews according to the project documentation
Input text column	review
Cleaned text column	clean_review
Labels	Positive and negative sentiment
Main task	Keyword extraction from movie reviews

4. System Pipeline

The system is organized as a sequence of NLP steps. Each step prepares data or produces outputs that are used by the following step.

Stage	Main file or method	Purpose
1	preprocessing.py	Clean the raw review text and save a cleaned dataset.
2	TF_IDF.py	Build a TF-IDF matrix and extract the top keywords for each review.
3	word_embeddings.py	Train Word2Vec and create document and keyword embeddings.

4	advanced_keyword_extraction.py	Apply cosine re-ranking and KMeans clustering.
5	evaluation.py	Evaluate model outputs against manual reference keywords.
Bonus	TextRank_model.py	Extract keywords using a graph-based TextRank approach.
Interface	gui.py	Provide a simple graphical interface for running and testing the project.

5. Text Preprocessing

Preprocessing is used to make the text cleaner and easier for the keyword extraction models to analyze. Raw reviews may contain uppercase letters, HTML tags, punctuation, numbers and common words that do not carry strong meaning. The preprocessing step removes or standardizes these elements.

Step	Description	Reason
Lowercasing	Convert all text to lowercase.	Treat words such as Movie and movie as the same word.
HTML removal	Remove HTML tags from reviews.	Remove formatting noise such as line break tags.
Punctuation and number removal	Keep only alphabetic text.	Reduce unnecessary vocabulary noise.
Tokenization	Split text into separate words.	Prepare text for stopword removal and feature extraction.
Stopword removal	Remove common English stopwords.	Remove words such as the, is, and and.
Saving output	Save the cleaned text as clean_review.	Create the input for the keyword extraction models.

6. TF-IDF Baseline Model

TF-IDF stands for Term Frequency-Inverse Document Frequency. It is a statistical method that measures how important a term is in a document compared with the entire collection of documents. A word receives a high TF-IDF score when it appears often in one document but does not appear too frequently across all documents.

In this project, TF-IDF is used as the baseline keyword extraction method. The model extracts both single words and two-word phrases, then selects the highest-scoring terms as keywords for each review.

Parameter	Value	Purpose
max_features	5000	Keep the top 5000 vocabulary features.
ngram_range	(1, 2)	Use unigrams and bigrams.
min_df	2	Ignore terms that appear in fewer than two documents.
max_df	0.85	Ignore terms that appear in more than 85% of documents.
top_n	10	Extract the top 10 keywords for each review.

The TF-IDF script saves two main files: tfidf_keywords_results.csv, which contains extracted keywords and scores and tfidf_feature_names.npy, which contains the vocabulary features used by the model.

7. Word2Vec Semantic Representation

TF-IDF is useful but it doesnot understand the meaning of words. Word2Vec is added to represent words as numerical vectors. Words that appear in similar contexts have similar vectors, which allows the system to compare keyword meaning with document meaning.

Configuration item	Value or description
Model	Word2Vec
Vector size	100
Window size	5
Minimum count	2
Epochs	10
Document embedding	Average of the word vectors in a review.
Keyword embedding	Average of the word vectors in a keyword or phrase.

8. Advanced Keyword Extraction Methods

8.1 Cosine Similarity Re-ranking

The cosine similarity method starts with TF-IDF candidate keywords. It then compares each keyword vector with the document vector. Keywords that are semantically closer to the overall document meaning are ranked higher. This method attempts to improve TF-IDF by adding meaning-based ranking.

8.2 KMeans Clustering

KMeans clustering groups candidate keyword embeddings into clusters. The system then selects representative keywords from different clusters. The purpose is to avoid returning many very similar keywords and instead produce a more diverse keyword list.

Method	Main advantage	Main limitation
Cosine re-ranking	Adds semantic similarity to keyword ranking.	Depends on the quality of Word2Vec embeddings.
KMeans clustering	Improves keyword diversity.	Cluster quality depends on embeddings and selected number of clusters.

9. TextRank Bonus Model

TextRank is a graph-based keyword extraction method inspired by PageRank. In this approach, words are represented as graph nodes and nearby words are connected with edges. Important words are identified based on graph centrality.

- It doesnot require a trained embedding model.
- It can work using word co-occurrence inside the document.
- It can form multi-word keyphrases from important adjacent words.
- It may be weaker for very short documents because there are fewer word relationships.

10. Evaluation Methodology

The project evaluates extracted keywords by comparing model outputs with manually selected reference keywords for 25 documents. The comparison uses standard information retrieval metrics.

Metric	Meaning
Precision	The fraction of extracted keywords that are correct.
Recall	The fraction of reference keywords that were successfully extracted.
F1-score	The harmonic mean of precision and recall.
TP	True positives: correctly extracted keywords.
FP	False positives: extracted keywords not found in the reference set.
FN	False negatives: reference keywords missed by the model.

11. Evaluation Results

The available evaluation results show that TF-IDF is the best-performing evaluated method. This means the simple statistical baseline produced the highest average F1-score for the manually annotated sample.

Model	Average F1-score	Interpretation
TF-IDF Baseline	0.189	Best result in the provided evaluation file.
Cosine Re-ranking	0.148	Adds semantic ranking but gives lower exact-match F1.
KMeans Clustering	0.152	Improves diversity but does not beat TF-IDF in the sample.

Sentiment	TF-IDF F1	Cosine F1	KMeans F1
Positive	0.213	0.163	0.158
Negative	0.139	0.115	0.137

12. Discussion of Results

The results show that TF-IDF remains a strong baseline for keyword extraction. Although it does not understand word meaning, it works well because it identifies terms that are distinctive within each review.

The semantic methods are theoretically useful, but they may not perform better under strict exact-match evaluation. For example, if the manual reference contains the word movie but the model extracts film, the output may be semantically correct but still counted as incorrect.

- Exact-match evaluation is strict and does not reward synonyms.
- Word2Vec quality may be limited when trained only on the project corpus.
- Rare names, movie titles, and domain-specific phrases may have weak embeddings.
- KMeans can improve diversity but diversity does not always increase exact keyword overlap.
- Positive reviews achieved better F1-scores than negative reviews in the available results.

13. GUI Component

The project includes a graphical user interface built with Tkinter. The GUI makes the system easier to run and demonstrate, especially for users who do not want to use command-line scripts.

GUI section	Function
Pipeline Runner	Runs preprocessing, TF-IDF, Word2Vec, advanced extraction, and evaluation.
Keyword Extractor	Allows a user to enter text and extract keywords.
Results Viewer	Displays output files and evaluation charts.
About	Shows information about the project and models.

14. Project Output Files

Output file	Purpose
IMDB_cleaned.csv	Cleaned dataset after preprocessing.
tfidf_keywords_results.csv	TF-IDF keywords and scores for each review.
tfidf_feature_names.npy	Saved TF-IDF vocabulary feature names.
word2vec_model.model	Trained Word2Vec model.
document_embeddings.npy	Document embedding vectors.
keyword_embeddings.pkl	Keyword embedding dictionary.
advanced_keywords_results.csv	Cosine and KMeans keyword extraction results.
evaluation_results.csv	Evaluation metrics for the manually annotated documents.
fig1 to fig9 PNG files	Visual summaries of evaluation results.
textrank_comparison.png	TextRank comparison visualization.

15. Strengths of the Project

- The project covers a complete NLP pipeline from preprocessing to evaluation.
- It compares several keyword extraction methods instead of relying on one model only.
- The code is organized into separate scripts for different stages.
- TF-IDF extracts both single words and two-word phrases.
- The project includes saved outputs, visualizations, and a GUI.
- Manual evaluation gives a real basis for comparing model performance.

16. Limitations

- The manual evaluation set contains only 25 documents, which is small compared with the full dataset.
- Exact-match scoring may mark meaningful synonyms as wrong.
- TF-IDF does not understand semantic meaning.
- Word2Vec embeddings trained only on the project dataset may not handle rare words well.
- The preprocessing pipeline does not appear to include lemmatization.
- Some file naming or setup details should be improved for reproducibility, such as adding a requirements.txt file.

17. Recommended Improvements

Improvement	Expected benefit
Increase the manual evaluation sample size.	Produces more reliable performance results.
Add lemmatization.	Groups related forms such as act, acts, acting, and actor.
Use semantic evaluation.	Gives partial credit to synonyms and related keywords.
Try pretrained embeddings.	Improves semantic representation and rare-word handling.
Use transformer-based models such as KeyBERT.	May improve semantic keyword quality.
Use noun phrase extraction.	Improves the quality of candidate keyphrases.
Add named entity recognition.	Improves extraction of actor names, movie titles, and places.
Add requirements.txt and relative paths.	Makes the project easier to run on other devices.

18. Conclusion

The Keyword Extraction project successfully demonstrates several important NLP techniques. It cleans raw movie reviews, extracts keywords using TF-IDF, adds semantic methods using Word2Vec, improves diversity with KMeans clustering, includes TextRank as a graph-based bonus method, and evaluates the outputs using standard metrics.

The available results show that TF-IDF is the strongest evaluated method in this setup. However, the project could be improved further by increasing the evaluation sample size, adding lemmatization, using semantic evaluation, and testing transformer-based keyword extraction methods. Overall, the project is well structured and suitable as a strong NLP coursework project.